

WEEK-04

Write a C program to simulate:

a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>

#define BUFFER_SIZE 5

int mutex = 1;
int full = 0;
int empty = BUFFER_SIZE;
int x = 0;

void producer() {
    if ((mutex == 1) && (empty != 0)) {
        mutex = 0;
        x++;
        full++;
        empty--;
        printf("Produced item %d\n", x);
        printf("empty=%d full=%d\n", empty, full);
        mutex = 1;
    } else {
        printf("Buffer Full!\n");
    }
}

void consumer() {
    if ((mutex == 1) && (full != 0)) {
        mutex = 0;
        printf("Consumed item %d\n", x);
        x--;
        full--;
        empty++;
        printf("empty=%d full=%d\n", empty, full);
        mutex = 1;
    } else {
        printf("Buffer Empty!\n");
    }
}

int main() {
    int choice;

    while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
```

```

        case 1: producer(); break;
        case 2: consumer(); break;
        case 3: return 0;
        default: printf("Invalid choice\n");
    }
}
}

```

Output:

```

"C:\Users\SHALINI K A\Desktop" x + v
1. Produce
2. Consume
3. Exit
Enter choice: 1
Produced item 1
empty=4 full=1

1. Produce
2. Consume
3. Exit
Enter choice: 1
Produced item 2
empty=3 full=2

1. Produce
2. Consume
3. Exit
Enter choice: 2
Consumed item 2
empty=4 full=1

```

WEEK-04

Write a C program to simulate
a) Producer - Consumer problem using semaphores

```

#include <stdio.h>
#define BUFFER_SIZE 5

int mutex = 1;
int full = 0;
int empty = BUFFER_SIZE;
int x = 0;

void producer() {
    while (1) {
        if ((mutex == 1) && (empty != 0)) {
            mutex = 0;
            x++;
            full++;
            empty--;
            printf("Produced item %d\n", x);
            printf("empty = %d full = %d\n", empty, full);
            mutex = 1;
        }
        else {
            printf("Buffer Full\n");
        }
    }
}

void consumer() {
    while (1) {
        if ((mutex == 1) && (full != 0)) {
            mutex = 0;
            printf("Consumed item %d\n", x);
            x--;
        }
    }
}

```

Page No. _____
Date: _____

```

    bull--;
    empty++;
    printf("empty = %d, bull = %d\n",
           empty, bull);
    mutex = 1;
}
else {
    printf("Buffer empty\n");
}
}

3.
int main() {
    int choice;
    while (1) {
        printf("1. Produce\n2. Consume\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: Produce(); break;
            case 2: Consume(); break;
            case 3: return 0;
            default: printf("Invalid choice\n");
        }
    }
}

```

Output

```

1. Produce
2. Consume
3. Exit
Enter choice: 1
Produced Item 1

```

Page No. _____
Date: _____

```

empty = 4    bull = 1

```

```

1. Produce
2. Consume
3. Exit
Enter choice: 1
Produced Item 2
empty = 3    bull = 2

```

```

1. Produce
2. Consume
3. Exit
Enter choice: 2
Consumed Item 2
empty = 4    bull = 1

```

[Signature]
24/4

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t forks[N];

void *philosopher(void *num)
{
    int id = *(int *)num;

    while (1)
    {
        printf("Philosopher %d is thinking...\n", id);
        sleep(1);

        sem_wait(&forks[id]);
        printf("Philosopher %d picked up left fork %d\n", id, id);

        sem_wait(&forks[(id + 1) % N]);
        printf("Philosopher %d picked up right fork %d\n",
            id, (id + 1) % N);

        printf("Philosopher %d is eating...\n", id);
        sleep(2);

        sem_post(&forks[(id + 1) % N]);
        printf("Philosopher %d released right fork %d\n",
            id, (id + 1) % N);

        sem_post(&forks[id]);
        printf("Philosopher %d released left fork %d\n", id, id);

        printf("-----\n");
    }
}

int main()
{
    pthread_t phil[N];
    int ids[N];
    for (int i = 0; i < N; i++)
    {
        sem_init(&forks[i], 0, 1);
    }
    for (int i = 0; i < N; i++)
    {
        ids[i] = i;
```

```

        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++)
    {
        pthread_join(phil[i], NULL);
    }
    for (int i = 0; i < N; i++)
    {
        sem_destroy(&forks[i]);
    }
    return 0;
}

```

Output:

```

Philosopher 3 is thinking...
Philosopher 0 is thinking...
Philosopher 4 is thinking...
Philosopher 2 is thinking...
Philosopher 1 is thinking...
Philosopher 4 picked up left fork 4
Philosopher 0 picked up left fork 0
Philosopher 1 picked up left fork 1
Philosopher 2 picked up left fork 2
Philosopher 3 picked up left fork 3

```

Page No. _____
Date: _____

```

sem_post(&forks[id]);
printf("Philosopher %d released left\n", id, id);
printf("-----\n");
}
}
int main() {
    pthread_t phil[N];
    int forks[N];
    for (int i=0; i<N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i=0; i<N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL,
            philosopher, &ids[i]);
    }
    for (int i=0; i<N; i++) {
        pthread_join(phil[i], NULL);
    }
    for (int i=0; i<N; i++) {
        sem_destroy(&forks[i]);
    }
    return 0;
}

```

Output

```

Philosopher 0 is thinking...
Philosopher 1 is thinking...
Philosopher 0 picked up left fork 0
Philosopher 0 picked up right fork 1
Philosopher 0 is eating...
Philosopher 0 released right fork 1
Philosopher 0 released left fork 0

```

Page No. _____
Date: _____

```

sem_post(&forks[id]);
printf("Philosopher %d released left\n", id, id);
printf("-----\n");
}
}
int main() {
    pthread_t phil[N];
    int forks[N];
    for (int i=0; i<N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i=0; i<N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL,
            philosopher, &ids[i]);
    }
    for (int i=0; i<N; i++) {
        pthread_join(phil[i], NULL);
    }
    for (int i=0; i<N; i++) {
        sem_destroy(&forks[i]);
    }
    return 0;
}

```

Output

```

Philosopher 0 is thinking...
Philosopher 1 is thinking...
Philosopher 0 picked up left fork 0
Philosopher 0 picked up right fork 1
Philosopher 0 is eating...
Philosopher 0 released right fork 1
Philosopher 0 released left fork 0

```

Reader - writer Problem

```
#include <stdio.h>
int main() {
    int readers = 5, writers = 3;
    for (int p=1; p<=readers; p++) {
        printf("Reader %d is reading data\n", p);
    }
    for (int p=1; p<=writers; p++) {
        printf("Writer %d is writing data\n", p);
    }
    return 0;
}
```

Output

Reader 1 is reading data
 Reader 2 is reading data
 Reader 3 is reading data
 Reader 4 is reading data
 Reader 5 is reading data
 Writer 1 is writing data
 Writer 2 is writing data